

GANDAKI UNIVERSITY

Faculty of Science and Technology
Bachelor of Computer Application (BCA)

WEB TECHNOLOGY

CHAPTER 3

INTRODUCING CASCADING STYLE SHEETS (CSS)

Complete Theory, Code Examples & Exam Q&A

Prepared by:

Mr. Dammar Khadayat

Lecturer, Department of Science and Technology
Gandaki University, Pokhara, Gandaki Pradesh, Nepal

Academic Year: 2081/82 (2024/25) | Total: 8 Hours

3.1 Introduction to CSS

What is CSS?

CSS stands for Cascading Style Sheets. It is a stylesheet language used to describe the presentation (visual appearance and layout) of HTML documents. CSS controls how HTML elements are displayed on screen, in print, or in other media. While HTML defines the structure and content, CSS defines the style and layout — colors, fonts, spacing, positioning, and responsiveness.

CSS was created by Håkon Wium Lie and Bert Bos. The first CSS specification (CSS1) was published by W3C in December 1996. CSS2 followed in 1998, and CSS3 (the current standard, delivered in modules) has been developed since 1999 and is still evolving.

Why CSS is Important:

- Separation of Concerns: HTML handles structure, CSS handles presentation. This makes code cleaner and easier to maintain.
- Consistency: One CSS file can style hundreds of HTML pages uniformly.
- Efficiency: Change one CSS rule to update all pages instantly.
- Accessibility: CSS enables better font sizes, contrast, and layouts for users with disabilities.
- Responsive Design: CSS media queries allow web pages to adapt to different screen sizes (desktop, tablet, mobile).
- Rich Visual Design: CSS enables animations, transitions, gradients, shadows, and complex layouts.

CSS Analogy

Think of a web page like a house:

HTML is the structure — walls, floors, rooms, windows.

CSS is the interior design — paint colors, furniture style, lighting, carpet, decorations.

Just as interior design doesn't change the structure of a house, CSS doesn't change the content — it only changes how it looks.

3.1.1 CSS Syntax

CSS Syntax Rule

A CSS rule consists of a selector and a declaration block. The selector targets the HTML element(s) to style. The declaration block (inside curly braces {}) contains one or more property: value pairs, each ending with a semicolon.

```

selector {
  property: value;
  property: value;
}

/* Example: */
h1 {
  color: #1F3864;           /* font color */
  font-size: 36px;          /* font size */
  font-family: Arial;       /* font type */
  text-align: center;       /* horizontal alignment */
  text-decoration: underline; /* underline */
}

p {
  color: #333333;
  font-size: 16px;
  line-height: 1.6;         /* space between lines */
  margin-bottom: 10px;
}

```

Parts of a CSS Rule:

Part	Description	Example
Selector	Identifies which HTML element(s) to style	h1, p, .classname, #idname
Declaration Block	The pair of curly braces { } containing all declarations	{ color: red; font-size: 16px; }
Property	The CSS attribute to change (what to style)	color, font-size, background-color
Value	The setting applied to the property	red, 16px, Arial, #FF0000
Declaration	One property: value pair	color: blue;
Semicolon	Separates declarations inside the block	color: red; (must end with ;)
Comment	Notes ignored by browser — /* comment */	/* This is a comment */

3.1.2 Where You Can Add CSS Rules — Three Methods

CSS can be added to HTML documents in three ways: Inline, Internal (Embedded), and External. Understanding when and why to use each is critical.

Method 1: Inline CSS

Inline CSS is applied directly to a specific HTML element using the style attribute inside the opening tag. It overrides all other CSS rules for that element.

```
<h1 style='color: darkblue; font-size: 36px; text-align: center;'>
  Welcome to Gandaki University
</h1>

<p style='color: red; font-weight: bold; background-color: yellow;'>
  This paragraph has inline styling.
</p>

<table style='border: 2px solid black; width: 100%; border-collapse:
  ...
</table>
```

Method 2: Internal (Embedded) CSS

Internal CSS is placed inside a <style> tag within the <head> section of the HTML document. It applies to the entire page but not to other pages.

```
<!DOCTYPE html>
<html>
<head>
  <title>Gandaki University</title>
  <style>
    /* Internal CSS – applies to entire page */
    body {
      font-family: Arial, sans-serif;
      background-color: #f0f4f8;
      margin: 0;
      padding: 0;
    }
    h1 {
      color: #1f3864;
      text-align: center;
      font-size: 32px;
    }
    p {
      color: #333;
      font-size: 16px;
      line-height: 1.8;
    }
    .highlight {
      background-color: yellow;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h1>Web Technology Notes</h1>
  <p>This is styled with <span class='highlight'>internal CSS</span>.</p>
</body>
</html>
```

Method 3: External CSS

External CSS is written in a separate .css file and linked to HTML pages using the <link> tag in the <head>. This is the BEST and MOST RECOMMENDED practice for real web development.

```
<!-- HTML file: index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Gandaki University</title>
  <link rel='stylesheet' type='text/css' href='styles.css'>
</head>
<body>
  <h1>Welcome</h1>
  <p>Styled by external CSS file.</p>
</body>
</html>

/* External file: styles.css */
body {
  font-family: 'Trebuchet MS', Arial, sans-serif;
  background-color: #f4f7f9;
  margin: 20px;
}
h1 {
  color: #1F3864;
  border-bottom: 3px solid #028090;
  padding-bottom: 8px;
}
p {
  color: #444;
  line-height: 1.7;
}
```

Internal CSS	External CSS
Placed in <style> inside <head>	Placed in a separate .css file
Applies to one HTML page only	Applies to all linked HTML pages
No extra HTTP request needed	Requires an HTTP request to load the .css file
Good for single-page quick styling	Best practice for multi-page websites
Changes require editing every HTML file	One file change updates the entire website
Harder to maintain on large sites	Easy to maintain and update

Inline CSS	Internal/External CSS
Highest specificity — overrides all other styles	Lower specificity but better for maintainability
Applies to ONE element only	Can target many elements at once

Inline CSS	Internal/External CSS
Mixes content with presentation — bad practice	Separates content from presentation — good practice
Very hard to maintain	Easy to update and maintain
Useful for quick tests or dynamic JS styling	Preferred for real development

3.1.3 Levels / Cascading Order of Style Sheets

The word 'Cascading' in CSS means that styles can come from multiple sources and they cascade (flow down) in a specific priority order. When conflicts occur, the browser resolves them using specificity and order rules.

The Cascade — Priority Order (High to Low):

1. Inline styles — style attribute on element (highest priority)
2. Internal styles — <style> in <head>
3. External styles — linked .css file
4. Browser default styles (lowest priority — user agent stylesheet)

What 'Cascading' Really Means

When multiple CSS rules target the same element with conflicting values, the browser applies a cascade to decide which rule wins.

Rule 1: Specificity — more specific selectors win over less specific ones.

Rule 2: Order — if specificity is equal, the rule declared LAST wins.

Rule 3: Importance — !important overrides everything (use sparingly).

Example: If you have `h1 { color: blue }` in external CSS and `h1 { color: red }` in internal CSS, the internal CSS wins (higher priority).

3.2 CSS Selectors, Properties, Colors, Pseudo-Classes, Positioning & Layout

3.2.1 CSS Selectors — Complete Guide

What is a CSS Selector?

A CSS selector is a pattern used to select and target specific HTML elements for styling. CSS provides many types of selectors — from simple element selectors to complex combinators and attribute selectors. Mastering selectors is essential for precise and efficient CSS.

A. Element (Type) Selector

Selects all HTML elements of a specific type/tag name. Applies to every instance of that tag on the page.

```
/* Styles ALL <p> elements on the page */
p {
  color: #333333;
  font-size: 16px;
  line-height: 1.7;
}

/* Styles ALL <h2> elements */
h2 {
  color: #1F3864;
  font-size: 24px;
  margin-bottom: 10px;
}

/* Styles ALL <a> (link) elements */
a {
  color: #0070c0;
  text-decoration: none; /* removes underline */
}
```

B. Class Selector

Targets elements with a specific class attribute. Classes can be reused on multiple elements. Use a dot (.) before the class name in CSS.

```
/* CSS */
.highlight {
  background-color: #FFFACD;
  font-weight: bold;
}
```

```
padding: 2px 6px;
}

.error-message {
  color: red;
  border: 1px solid red;
  padding: 8px;
  background-color: #fff0f0;
}

.btn {
  background-color: #1F3864;
  color: white;
  padding: 10px 20px;
  border-radius: 4px;
  cursor: pointer;
}

/* HTML usage: */
<p class='highlight'>This text is highlighted.</p>
<span class='error-message'>Invalid email address!</span>
<button class='btn'>Submit</button>
<div class='btn'>Fake Button</div>  <!-- both get same style -->
```

C. ID Selector

Targets a single element with a specific id attribute. IDs must be unique — only one element per page should have a given ID. Use a hash (#) before the ID name in CSS.

```
/* CSS */
#main-header {
  background-color: #1F3864;
  color: white;
  padding: 20px;
  text-align: center;
}

#footer {
  background-color: #333;
  color: #aaa;
  text-align: center;
  padding: 15px;
  font-size: 13px;
}

/* HTML usage: */
<header id='main-header'>
  <h1>Gandaki University</h1>
</header>
<footer id='footer'>
  &copy; 2024 Gandaki University
</footer>
```

Class Selector (.)	ID Selector (#)
Targets elements with class='name'	Targets element with id='name'
Can be used on multiple elements	Must be unique — only ONE element per ID per page
Syntax: .classname { }	Syntax: #idname { }
Lower specificity than ID	Higher specificity than class
Reusable — apply same style to many elements	Not reusable — unique per element
Example: .btn, .card, .highlight	Example: #header, #nav, #footer

D. Universal Selector

```
/* Selects ALL elements on the page */
* {
  margin: 0;      /* reset default margin */
  padding: 0;     /* reset default padding */
  box-sizing: border-box; /* include padding in width */
}
```

E. Grouping Selector

Apply the same style to multiple selectors by separating them with commas.

```
/* Instead of writing separate rules: */
h1 { color: #1F3864; font-family: Arial; }
h2 { color: #1F3864; font-family: Arial; }
h3 { color: #1F3864; font-family: Arial; }

/* Use a grouping selector: */
h1, h2, h3 {
  color: #1F3864;
  font-family: Arial;
}

/* Mix different selectors: */
h1, .intro-text, #page-title {
  font-weight: bold;
  text-transform: uppercase;
}
```

F. Descendant Selector

Targets elements that are nested (descendant) inside another element. Separated by a space.

```
/* Any <p> inside a <div> */
div p {
    color: navy;
    padding-left: 20px;
}

/* Any <a> inside <nav> */
nav a {
    color: white;
    text-decoration: none;
    margin-right: 15px;
}

/* Any <li> inside .sidebar */
.sidebar li {
    border-bottom: 1px solid #ccc;
    padding: 5px 0;
}
```

G. Attribute Selector

```
/* Elements with a specific attribute */
input[type='text'] { border: 2px solid #1F3864; }
input[type='submit'] { background-color: #028090; color: white; }
a[target='_blank'] { color: orange; } /* links opening in new tab */
img[alt] { border: 2px solid green; } /* images WITH alt text */
```

3.2.2 CSS Style Specification Formats

CSS supports multiple formats for specifying values for properties like color, size, and font.

A. Color Specification Formats

```
/* 1. Named Colors (140+ predefined names) */
color: red;
color: blue;
color: darkslateblue;
color: tomato;
color: coral;
background-color: lightyellow;

/* 2. Hexadecimal (most common in web dev) */
color: #FF0000; /* red — #RRGGBB format */
color: #00FF00; /* green */
color: #0000FF; /* blue */
color: #1F3864; /* dark navy blue */
color: #028090; /* teal */
color: #FFF; /* white (shorthand: #RGB) */
color: #000; /* black */
```

```

/* 3. RGB — Red, Green, Blue (0-255) */
color: rgb(255, 0, 0);      /* red */
color: rgb(31, 56, 100);    /* #1F3864 navy */
color: rgb(0, 0, 0);        /* black */
color: rgb(255, 255, 255);  /* white */

/* 4. RGBA — RGB + Alpha (transparency: 0.0=transparent, 1.0=opaque) */
color: rgba(255, 0, 0, 1.0); /* solid red */
color: rgba(0, 0, 0, 0.5);   /* 50% transparent black */
background-color: rgba(31, 56, 100, 0.8); /* semi-transparent navy */

/* 5. HSL — Hue (0-360), Saturation (0-100%), Lightness (0-100%) */
color: hsl(220, 53%, 26%);   /* navy blue equivalent */
color: hsl(0, 100%, 50%);    /* red */
color: hsl(120, 100%, 50%);  /* green */

/* 6. HSLA — HSL + Alpha */
color: hsla(220, 53%, 26%, 0.7); /* 70% opaque navy */

```

B. Size / Length Units

Unit	Type	Description	Example
px	Absolute	Pixels — fixed size, most precise	font-size: 16px
pt	Absolute	Points — 1pt = 1.33px. Used for print	font-size: 12pt
em	Relative	Relative to parent's font-size. 1em = parent's font-size	font-size: 1.5em (1.5x parent)
rem	Relative	Relative to root (<html>) font-size. More predictable than em	font-size: 1.2rem
%	Relative	Percentage of parent element's size	width: 80%
vw	Viewport	1vw = 1% of viewport width	width: 100vw (full width)
vh	Viewport	1vh = 1% of viewport height	height: 100vh (full height)
cm / mm	Absolute	Centimeters / Millimeters — used for print	width: 10cm

3.2.3 CSS Style Classes — Using class Attribute

CSS classes are one of the most powerful features of CSS. A class defines a reusable set of styles that can be applied to any HTML element using the class attribute.

```

/* CSS Classes Example — Card Layout */

```

```
.card {
  background-color: white;
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 20px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
  margin: 10px;
  width: 300px;
}

.card-title {
  font-size: 20px;
  font-weight: bold;
  color: #1F3864;
  margin-bottom: 10px;
}

.card-body {
  font-size: 15px;
  color: #555;
  line-height: 1.6;
}

/* Multiple classes on one element */
.text-center { text-align: center; }
.text-red    { color: red; }
.bold        { font-weight: bold; }

/* HTML: apply multiple classes */
<p class='text-center text-red bold'>Error: Invalid Input!</p>
```

3.2.4 Important CSS Properties and Values

A. Typography Properties

```
p {
  font-family: Arial, Helvetica, sans-serif; /* Font stack */
  font-size: 16px;
  font-weight: bold; /* normal, bold, 100-900 */
  font-style: italic; /* normal, italic, oblique */
  font-variant: small-caps;
  line-height: 1.6; /* space between lines */
  letter-spacing: 2px; /* space between characters */
  word-spacing: 5px; /* space between words */
  text-align: justify; /* left, right, center, justify */
  text-decoration: underline; /* none, underline, line-through, overline */
  text-transform: uppercase; /* lowercase, uppercase, capitalize */
  text-indent: 30px; /* first line indent */
  color: #333333;
}
```

B. Background Properties

```
body {
    background-color: #f0f4f8;          /* Solid color */
    background-image: url('bg.jpg');    /* Background image */
    background-repeat: no-repeat;       /* repeat, repeat-x, repeat-y, no-repeat */
    background-position: center top;    /* position of image */
    background-size: cover;             /* cover, contain, auto, 100% */
    background-attachment: fixed;       /* scroll, fixed */
}

/* Shorthand: */
body {
    background: #1F3864 url('banner.jpg') no-repeat center/cover fixed;
}
```

C. Box Model Properties

The CSS Box Model

Every HTML element is treated as a rectangular box. The box model describes the layers around content:

Content → Padding → Border → Margin

Content: The actual text/image.

Padding: Space between content and border (inside the box). Has the background color.

Border: The visible edge of the element. Can have style, width, color.

Margin: Space OUTSIDE the border, between this element and others. Always transparent.

```
.box {
    /* Content size */
    width: 300px;
    height: 150px;

    /* Padding - inner spacing (all 4 sides) */
    padding: 20px;          /* all sides */
    padding: 10px 20px;     /* top-bottom | left-right */
    padding: 5px 10px 15px 20px; /* top | right | bottom | left (clockwise) */
    padding-top: 10px;      /* individual sides */

    /* Border */
    border: 2px solid #1F3864;
    border-width: 1px;
    border-style: solid;     /* solid, dashed, dotted, double, none */
    border-color: #028090;
    border-radius: 8px;      /* rounded corners */
    border-top: 3px dashed red; /* individual side */
}
```

```

/* Margin – outer spacing */
margin: 20px;           /* all sides */
margin: 10px auto;      /* top-bottom=10px, left-right=auto (centers
element) */
margin-left: 30px;

/* Box sizing */
box-sizing: border-box; /* width includes padding + border */
}

```

D. Display and Visibility Properties

```

.item {
  display: block;           /* Takes full width, starts on new line */
  display: inline;          /* Flows with text, no width/height */
  display: inline-block;    /* Like inline but allows width/height */
  display: none;            /* Hides element AND removes from layout */
  display: flex;            /* Flexbox layout */
  display: grid;            /* Grid layout */

  visibility: visible;      /* Default */
  visibility: hidden;       /* Hides but KEEPS space in layout */

  opacity: 1.0;             /* 0.0 = transparent, 1.0 = opaque */
  opacity: 0.5;            /* 50% transparent */
}

```

display: none	visibility: hidden
Element is completely removed from the page layout	Element is hidden but its space is still reserved
Other elements fill the space it occupied	A blank space remains where the element was
The element cannot be interacted with	Element cannot be seen but affects layout
Often used with JavaScript to toggle content	Less commonly used for toggle effects

3.2.5 :focus and :active Pseudo-Classes

What are Pseudo-Classes?

A CSS pseudo-class is a keyword added to a selector that specifies a special STATE of an element. Pseudo-classes allow you to style elements dynamically based on user interaction, position, or state — without JavaScript.

Syntax: selector:pseudo-class { declarations }

A. :link, :visited, :hover, :active — Link Pseudo-Classes

```
/* Style for unvisited links */
a:link {
    color: #0070c0;
    text-decoration: none;
}

/* Style for visited links */
a:visited {
    color: #7030a0;
}

/* Style when mouse hovers over the link */
a:hover {
    color: #C0392B;
    text-decoration: underline;
    background-color: #FFF3CD;
}

/* Style at the moment of clicking (mouse button pressed) */
a:active {
    color: #FF0000;
    font-weight: bold;
}

/* Important: Order must be L-V-H-A (LoVe HAtE) */
/* :link, :visited, :hover, :active */
```

B. :focus Pseudo-Class

The `:focus` pseudo-class applies when an element receives focus — typically when a user clicks on it or tabs to it. Most commonly used for form inputs.

```
/* Style input when it receives focus */
input:focus {
    border: 2px solid #028090;
    background-color: #E0F7FA;
    outline: none; /* remove default browser outline */
    box-shadow: 0 0 5px rgba(2,128,144,0.5);
}

textarea:focus {
    border-color: #1F3864;
    outline: none;
}

button:focus {
    outline: 2px dashed #028090;
    outline-offset: 3px;
}
```

C. :active Pseudo-Class

The :active pseudo-class applies while an element is being activated — the brief moment the user presses and holds the mouse button on a clickable element.

```
button:active {
  background-color: #0a1f3d; /* darker shade when pressed */
  transform: translateY(2px); /* slight downward movement effect */
  box-shadow: none;
}

a:active {
  color: darkred;
}
```

D. Other Useful Pseudo-Classes

Pseudo-Class	When It Applies	Example
:hover	Mouse pointer is over the element	button:hover { background: teal; }
:focus	Element has keyboard/click focus	input:focus { border: 2px solid blue; }
:active	Element is being clicked	a:active { color: red; }
:visited	Link has been visited	a:visited { color: purple; }
:first-child	Element is the first child of its parent	li:first-child { font-weight: bold; }
:last-child	Element is the last child of its parent	li:last-child { border: none; }
:nth-child(n)	Element is the nth child (or odd/even)	tr:nth-child(even) { background: #f2f2f2; }
:disabled	Form element is disabled	input:disabled { background: #eee; }
:checked	Checkbox or radio button is checked	input:checked { accent-color: teal; }
:not(selector)	Elements NOT matching selector	:not(p) { display: flex; }

3.2.6 CSS Positioning

CSS Position Property

The CSS position property controls how an element is placed in the document layout. It determines whether an element flows normally, is removed from the flow, or is positioned relative to something specific. The position property, combined with top, right, bottom, left values, places elements precisely on the page.

Value	Description	Behavior
static	Default value for all elements	Element flows in normal document flow. top/left/right/bottom have NO effect.
relative	Positioned relative to its NORMAL position	Element stays in normal flow but can be offset with top/left/right/bottom. Space is still reserved.
absolute	Positioned relative to nearest positioned ancestor	Removed from normal flow. Positioned relative to nearest parent with position $\neq$ static. Space NOT reserved.
fixed	Positioned relative to the BROWSER WINDOW	Removed from flow. Stays in same spot even when page scrolls. Used for sticky headers/navbars.
sticky	Hybrid of relative and fixed	Acts like relative until scrolled to a threshold, then acts like fixed. Used for sticky table headers.

```
/* RELATIVE POSITIONING */
.relative-box {
  position: relative;
  top: 20px;      /* move 20px DOWN from normal position */
  left: 30px;     /* move 30px RIGHT from normal position */
  background-color: lightyellow;
}

/* ABSOLUTE POSITIONING */
.parent-container {
  position: relative; /* makes this the reference for absolute child */
  width: 400px;
  height: 300px;
  background-color: #e0e0e0;
}

.absolute-badge {
  position: absolute;
  top: 10px;      /* 10px from top of parent */
  right: 10px;    /* 10px from right of parent */
  background-color: red;
  color: white;
```

```
padding: 4px 8px;
border-radius: 50%;
}

/* FIXED POSITIONING — stays on screen during scroll */
nav.sticky-navbar {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  background-color: #1F3864;
  color: white;
  padding: 15px 20px;
  z-index: 1000; /* appears above other elements */
}

/* STICKY POSITIONING */
thead th {
  position: sticky;
  top: 0;
  background-color: #1F3864;
  color: white;
}
```

Z-Index — Controlling Stack Order

When elements overlap due to positioning, z-index controls which element appears on top. Higher z-index = closer to the user.

```
.modal-overlay {
  position: fixed;
  z-index: 999; /* below modal */
  background: rgba(0,0,0,0.5);
  width: 100%; height: 100%;
  top: 0; left: 0;
}

.modal-box {
  position: fixed;
  z-index: 1000; /* on top of overlay */
  background: white;
  padding: 30px;
  top: 50%; left: 50%;
  transform: translate(-50%, -50%);
}
```

3.2.7 Layout with CSS — Float and Flexbox

A. Float Layout (Traditional)

The float property was historically used to create multi-column layouts. It takes an element out of normal flow and places it to the left or right of its container, allowing other content to wrap around it.

```
.left-column {
  float: left;
  width: 30%;
  padding: 10px;
  background-color: #EBF3FF;
}

.right-column {
  float: right;
  width: 65%;
  padding: 10px;
}

/* Clear fix – prevents parent collapse */
.container::after {
  content: '';
  display: table;
  clear: both;
}
```

B. Flexbox Layout (Modern)

CSS Flexbox (Flexible Box Layout) is a modern layout model that makes it easy to design flexible and responsive layout structures without using float or positioning.

```
.navbar {
  display: flex;
  flex-direction: row;          /* row (default), column, row-reverse */
  justify-content: space-between; /* main axis alignment */
  align-items: center;          /* cross axis alignment */
  background-color: #1F3864;
  padding: 10px 20px;
}

.card-container {
  display: flex;
  flex-wrap: wrap;              /* allows items to wrap to next line */
  gap: 20px;                    /* space between items */
  justify-content: center;
}

.card {
  flex: 1 1 280px; /* grow, shrink, basis – min 280px each */
  background: white;
  border-radius: 8px;
  padding: 20px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
```

3.3 Complete CSS Program with Different Kinds of Styles

The following is a complete, fully-styled web page demonstrating all major CSS concepts: selectors, colors, typography, box model, positioning, hover effects, and layout.

```
<!DOCTYPE html>
<html lang='en'>
<head>
  <meta charset='UTF-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1.0'>
  <title>CSS Demonstration – Gandaki University</title>
  <style>

    /* ===== RESET & BASE ===== */
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: Arial, Helvetica, sans-serif;
      background-color: #f0f4f8;
      color: #333;
    }

    /* ===== NAVIGATION BAR – fixed ===== */
    nav {
      position: fixed;
      top: 0; left: 0; width: 100%;
      background-color: #1f3864;
      padding: 12px 24px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      z-index: 1000;
    }
    nav .logo { color: white; font-size: 20px; font-weight: bold; }
    nav a {
      color: #cadcf8;
      text-decoration: none;
      margin-left: 20px;
      font-size: 15px;
    }
    nav a:hover { color: white; text-decoration: underline; }

    /* ===== HERO SECTION ===== */
    .hero {
      margin-top: 56px;
      background-color: #028090;
      color: white;
      text-align: center;
      padding: 60px 20px;
    }
```

```
}
.hero h1 { font-size: 42px; margin-bottom: 12px; }
.hero p { font-size: 18px; opacity: 0.9; }

/* ===== MAIN CONTENT ===== */
main { max-width: 1100px; margin: 30px auto; padding: 0 20px; }

/* ===== CARDS (Flexbox) ===== */
.card-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  margin-bottom: 30px;
}
.card {
  flex: 1 1 280px;
  background-color: white;
  border-radius: 8px;
  padding: 20px;
  border-top: 4px solid #028090;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  transition: box-shadow 0.3s ease;
}
.card:hover {
  box-shadow: 0 6px 20px rgba(0,0,0,0.15);
  transform: translateY(-2px);
}
.card h3 { color: #1F3864; margin-bottom: 10px; }
.card p { color: #666; line-height: 1.6; font-size: 14px; }

/* ===== TABLE ===== */
table { width: 100%; border-collapse: collapse; margin-bottom: 30px; }
th {
  background-color: #1F3864;
  color: white;
  padding: 12px 15px;
  text-align: left;
}
td { padding: 10px 15px; border-bottom: 1px solid #ddd; }
tr:nth-child(even) td { background-color: #EBF3FF; }
tr:hover td { background-color: #cce5ff; }

/* ===== FORM ===== */
.form-section { background: white; padding: 24px; border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08); margin-bottom:
30px; }
.form-section h2 { color: #1F3864; margin-bottom: 16px; }
label { display: block; margin: 12px 0 4px; font-weight: bold; color:
#444; }
input[type='text'], input[type='email'], select, textarea {
  width: 100%; padding: 9px 12px;
  border: 1px solid #ccc; border-radius: 4px; font-size: 15px;
}
input[type='text']:focus, input[type='email']:focus,
select:focus, textarea:focus {
```

```
        border-color: #028090;
        outline: none;
        box-shadow: 0 0 0 3px rgba(2,128,144,0.15);
    }
    .btn-submit {
        margin-top: 14px;
        background-color: #028090;
        color: white; border: none;
        padding: 11px 28px; border-radius: 4px;
        font-size: 15px; cursor: pointer;
    }
    .btn-submit:hover { background-color: #026070; }
    .btn-submit:active { background-color: #014050; }

    /* ===== FOOTER ===== */
    footer {
        background-color: #1F3864;
        color: #CADCFD;
        text-align: center;
        padding: 20px;
        font-size: 14px;
    }

</style>
</head>
<body>

    <!-- Navigation -->
    <nav>
        <span class='logo'>Gandaki University</span>
        <div>
            <a href='#'>Home</a>
            <a href='#'>About</a>
            <a href='#'>Courses</a>
            <a href='#'>Contact</a>
        </div>
    </nav>

    <!-- Hero -->
    <div class='hero'>
        <h1>Web Technology with CSS</h1>
        <p>Chapter 3 – Cascading Style Sheets Demo</p>
    </div>

    <!-- Main Content -->
    <main>
        <div class='card-grid'>
            <div class='card'>
                <h3>Selectors</h3>
                <p>CSS selectors target HTML elements for styling.</p>
            </div>
            <div class='card'>
                <h3>Box Model</h3>
                <p>Padding, Border, Margin define spacing around elements.</p>
            </div>
        </div>
    </main>
</body>
</html>
```

```
</div>
<div class='card'>
  <h3>Flexbox</h3>
  <p>Modern layout system for responsive web design.</p>
</div>
</div>

<table>
  <thead>
    <tr><th>Subject</th><th>Teacher</th><th>Credit Hours</th></tr>
  </thead>
  <tbody>
    <tr><td>Web Technology</td><td>Mr. D.
Khadayat</td><td>3</td></tr>
    <tr><td>Database Systems</td><td>Mr. R.
Sharma</td><td>3</td></tr>
    <tr><td>Networking</td><td>Ms. S. Thapa</td><td>3</td></tr>
  </tbody>
</table>

<div class='form-section'>
  <h2>Contact Us</h2>
  <form>
    <label>Full Name:</label>
    <input type='text' placeholder='Enter your name' />
    <label>Email:</label>
    <input type='email' placeholder='you@example.com' />
    <label>Message:</label>
    <textarea rows='3' placeholder='Your message... '></textarea>
    <button class='btn-submit'>Send Message</button>
  </form>
</div>
</main>

<footer>
  &copy; 2024 Gandaki University, Pokhara, Nepal – Web Technology
Department
</footer>

</body>
</html>
```

EXAM QUESTIONS AND ANSWERS — Chapter 3: CSS

Cascading Style Sheets | Web Technology | Gandaki University

PART A — Short Answer Questions (2 Marks)

Q: What is CSS? Why is it used? [2 Marks]

Ans: CSS stands for Cascading Style Sheets. It is a language used to describe the visual presentation of HTML documents.

CSS is used to: (1) Control colors, fonts, spacing, and layout of web pages. (2) Separate structure (HTML) from presentation (CSS). (3) Apply consistent styles across multiple pages with one stylesheet. (4) Create responsive designs that work on different screen sizes. (5) Add animations, transitions, and interactive effects without JavaScript.

Q: What are the three ways to add CSS to an HTML document? Explain briefly. [2 Marks]

Ans: 1. Inline CSS: Written using the style attribute directly on an HTML tag. E.g.: `<p style='color: red;'>`. Highest priority but hardest to maintain.

2. Internal CSS: Written inside `<style>` tags in the `<head>` section. Applies to that page only.

3. External CSS: Written in a separate `.css` file and linked with `<link rel='stylesheet' href='style.css'>`. Best practice — applies to all linked pages. Most maintainable.

Q: What is the difference between class selector and ID selector in CSS? [2 Marks]

Ans: Class Selector (.): Uses a dot prefix. Applied with `class='name'` attribute. Can be used on multiple elements — reusable. Lower specificity. Example: `.highlight { color: red; }`

ID Selector (#): Uses a hash prefix. Applied with `id='name'` attribute. MUST be unique per page — only ONE element per ID. Higher specificity. Example: `#header { background: navy; }`

Use class for reusable styles, use ID for unique page elements.

Q: What is the CSS Box Model? Draw and explain. [2 Marks]

Ans: The CSS Box Model describes the rectangular space occupied by every HTML element. It consists of four layers from inside out:

1. Content: The actual text or image content. Width and height apply here.

2. Padding: Space between content and border. Has the element's background color.

3. Border: The visible boundary line around padding + content.

4. Margin: Space OUTSIDE the border, separating from neighboring elements. Always transparent.

Total element width = content width + left/right padding + left/right border + left/right margin.

Q: What is the difference between :focus and :active pseudo-classes? [2 Marks]

Ans: :focus — Applies when an element receives user focus, typically when a user clicks on a form input or tabs to it using the keyboard. Stays applied while the element is focused. Example: `input:focus { border: 2px solid blue; }`

:active — Applies only at the MOMENT of clicking — while the mouse button is held down. It reverts to normal as soon as the mouse button is released. Example: `button:active { background: darkblue; }`

Key difference: :focus persists while element is focused. :active is only during the click moment.

Q: What is the difference between position: relative and position: absolute? [2 Marks]

Ans: position: relative — Element stays in normal document flow (its original space is preserved). It can then be offset with top/left/right/bottom values relative to its OWN normal position.

position: absolute — Element is removed from normal flow (no space is reserved). It is positioned relative to its NEAREST ANCESTOR that has any position other than static. If no such ancestor, it positions relative to the <html> element.

Use relative as a container for absolute children. Absolute is commonly used for badges, tooltips, and dropdowns.

Q: What does 'Cascading' mean in CSS? Explain the priority order. [2 Marks]

Ans: 'Cascading' means that CSS rules can come from multiple sources and they overlap/cascade with specific priority rules.

Priority order (highest to lowest): 1. Inline styles (`style=""`) — highest. 2. Internal styles (`<style>` in `<head>`). 3. External styles (linked .css file). 4. Browser default styles — lowest.

When two rules conflict on the same element: (a) Higher priority source wins. (b) More specific selector wins (ID > class > element). (c) If same specificity, the LAST rule in code wins. (d) !important overrides everything.

Q: What is the :hover pseudo-class? Write an example. [2 Marks]

Ans: :hover is a CSS pseudo-class that applies styles to an element when the user moves the mouse pointer over it. It is used to create interactive visual effects like highlighting buttons, changing link colors, or showing dropdown menus.

Example: `button { background-color: #1F3864; color: white; padding: 10px 20px; } button:hover { background-color: #028090; cursor: pointer; }`

Common uses: Navigation menus that highlight on hover, buttons that change color, cards that show a shadow on hover, image galleries that show captions on hover.

Q: What is the difference between display: none and visibility: hidden? [2 Marks]

Ans: display: none — The element is completely removed from the page layout. It takes up NO space. Other elements fill the gap where it was. Useful for toggling content with JavaScript.
visibility: hidden — The element is invisible (not rendered) but it STILL OCCUPIES its space on the page. Other elements do NOT move to fill its space. A blank gap remains.
Think of it as: display:none = element doesn't exist. visibility:hidden = element is invisible but its seat is still taken.

Q: What are CSS pseudo-classes? Name any five with their purpose. [2 Marks]

Ans: CSS pseudo-classes are keywords added to selectors to style elements in a specific STATE or position, without adding extra classes to HTML.
:hover — when mouse is over the element. :focus — when form element has keyboard focus.
:active — while element is being clicked. :visited — for visited links. :first-child — first child of parent. :last-child — last child. :nth-child(n) — nth child (use even/odd for alternating rows).
:checked — checked checkboxes or radio buttons. :disabled — disabled form elements.

PART B — Long Answer Questions (5–10 Marks)

Q: Explain all three methods of adding CSS to HTML with examples. Which method is best and why? [5 Marks] [5 Marks]

Ans: Three CSS methods:

1. Inline CSS (style attribute): Written directly on the HTML tag. style='color:red;'. Highest priority. Hardest to maintain. Good only for temporary tests or dynamic JS styling.
 2. Internal CSS (<style> in <head>): Entire CSS in the HTML file inside <style>...</style>. Good for single-page demos. Cannot be shared across pages. Increases HTML file size.
 3. External CSS (<link> to .css file): Best practice. Written in a separate file (style.css). Linked with <link rel='stylesheet' href='style.css'> in <head>. Cached by browser — faster loading. One file styles hundreds of pages. Easy to maintain — change one file to update entire site.
- Best Method: External CSS — because it separates concerns (HTML = structure, CSS = style), enables code reuse, improves performance through browser caching, and is easiest to maintain on large websites.

Q: Explain CSS Selectors in detail: Element, Class, ID, Descendant, Pseudo-class, with examples for each. [5 Marks] [5 Marks]

Ans: 1. Element Selector: Targets all elements of a tag type. p { color: navy; } — styles ALL <p> elements.

2. Class Selector (.): Targets elements with a specific class attribute. `.btn { background: teal; }` — use `class='btn'` in HTML. Reusable on many elements.
3. ID Selector (#): Targets ONE unique element. `#header { background: navy; }` — use `id='header'`. Must be unique per page. Higher specificity.
4. Descendant Selector: Targets elements nested inside another. `nav a { color: white; }` — only `<a>` tags inside `<nav>`.
5. Grouping Selector: Applies same style to multiple selectors. `h1, h2, h3 { font-family: Arial; }` — comma-separated list.
6. Universal Selector: `* { margin: 0; }` — targets ALL elements.
7. Pseudo-class Selector: Targets elements in a state. `a:hover { color: red; }` — when mouse hovers over link.

Q: Explain the CSS Box Model in detail with a diagram description and code example. [5 Marks] [5 Marks]

Ans: The CSS Box Model treats every HTML element as a rectangular box with four layers:
Layer 1 — Content: The actual content (text, image). width and height properties apply here.
Layer 2 — Padding: Space between content and border. Has the element's background. `padding: 20px;` or `padding: top right bottom left;`
Layer 3 — Border: Visible edge around the element. `border: 2px solid #333;` Can set style (solid, dashed, dotted), width, color, and radius.
Layer 4 — Margin: Space OUTSIDE the border. Always transparent. `margin: 20px auto;` (auto centers block elements horizontally).
Total Width = content-width + padding-left + padding-right + border-left + border-right + margin-left + margin-right.
box-sizing: border-box; — makes width INCLUDE padding and border, simplifying layout calculations. This is the modern standard used by most developers.

Q: Explain CSS Positioning (static, relative, absolute, fixed, sticky) with examples. [10 Marks] [10 Marks]

Ans: CSS position property controls element placement:

1. static (default) — Normal document flow. top/left have no effect. All elements are static by default.
2. relative — Element stays in flow (space preserved) but can be offset with top/left/right/bottom relative to its OWN normal position. Used as positioning context for absolute children.
3. absolute — Removed from normal flow (no space reserved). Positioned relative to nearest positioned ancestor. If none, relative to `<html>`. Used for: badges, tooltips, popup menus.
4. fixed — Removed from flow. Positioned relative to BROWSER WINDOW. Does not move on scroll. Used for: sticky navigation bars, back-to-top buttons, fixed sidebars.
5. sticky — Hybrid. Acts like relative until scrolled past a threshold, then acts like fixed. Used for: sticky table headers, sticky section headings.

z-index controls which positioned elements appear on top. Higher z-index = front. Only works on elements with position $\neq$ static.

Q: Write a complete CSS program that demonstrates: navigation bar, responsive cards layout (Flexbox), a styled table with hover effects, and styled form with :focus styles. [10 Marks] [10 Marks]

Ans: A complete CSS program should demonstrate:

1. Navigation Bar: `position:fixed; display:flex; justify-content:space-between;` for sticky navbar.
 2. Flexbox Cards: `.container { display:flex; flex-wrap:wrap; gap:20px; }` for responsive card grid.
 3. Table Styling: `border-collapse:collapse;` for clean table. `tr:nth-child(even)` for alternating row colors. `tr:hover` for row highlight.
 4. Form with `:focus`: `input:focus { border-color: teal; box-shadow: 0 0 5px teal; outline:none; }` for visual feedback.
 5. Hover Effects: `button:hover { background: darker; }` `card:hover { box-shadow: larger; transform: translateY(-2px); }`
 6. Box Model: margin, padding, border-radius on all components.
- Refer to the complete code example in Section 3.3 of these notes for the full implementation.

PART C — Objective / MCQ Questions

Q#	Question	Answer
1	What does CSS stand for?	Cascading Style Sheets
2	Which HTML tag links an external CSS file?	<code><link rel='stylesheet' href='...'></code>
3	Which CSS method has the highest priority?	Inline CSS (style attribute)
4	Which selector targets elements with class='box'?	<code>.box { }</code>
5	Which selector targets an element with id='header'?	<code>#header { }</code>
6	Which selector targets ALL elements on the page?	<code>* { }</code> (Universal selector)
7	What is the correct CSS syntax?	<code>selector { property: value; }</code>
8	What color format is #FF0000?	Hexadecimal (red)
9	Which CSS property changes the text color?	color
10	Which property changes the background color?	background-color
11	Which property sets the font size?	font-size
12	Which property adds space INSIDE the element border?	padding
13	Which property adds space OUTSIDE the element border?	margin
14	<code>margin: 10px auto;</code> does what?	10px top/bottom, centers element left/right

Q#	Question	Answer
15	Which pseudo-class applies when mouse is over an element?	:hover
16	Which pseudo-class applies when an input is clicked/focused?	:focus
17	Which pseudo-class applies while a button is being clicked?	:active
18	Which pseudo-class styles visited links?	:visited
19	What order should link pseudo-classes be written?	L-V-H-A: :link :visited :hover :active
20	position: fixed positions element relative to what?	The browser window (viewport)
21	Which position value keeps the element in normal flow?	static (default) or relative
22	Which position removes element from flow entirely?	absolute or fixed
23	Which CSS property controls stacking order of positioned elements?	z-index
24	What does display: none do?	Hides element AND removes it from layout
25	What does visibility: hidden do?	Hides element but preserves its layout space
26	Which CSS property creates a Flexbox container?	display: flex
27	Which CSS unit is relative to parent font size?	em
28	Which CSS unit is relative to root font size?	rem
29	What does box-sizing: border-box do?	Includes padding+border in element's width
30	Which property makes corners rounded?	border-radius
31	tr:nth-child(even) selects which rows?	Even-numbered table rows (2nd, 4th, 6th...)
32	Which property controls transparency?	opacity (0.0 to 1.0)
33	rgba(0,0,0,0.5) means what?	Black color at 50% transparency
34	Which CSS module creates responsive layouts without float?	Flexbox (display: flex) or Grid (display: grid)
35	What CSS property sets space between text lines?	line-height

Chapter 3 — Summary & Quick Revision

Key Points to Remember

- CSS = Cascading Style Sheets. Controls visual presentation of HTML. Created by W3C.
- Three methods: Inline (highest priority) → Internal → External (best practice, lowest priority).
- CSS Syntax: selector { property: value; } — every declaration ends with semicolon.
- Selectors: Element (p), Class (.name), ID (#name), Universal (*), Group (a,b,c), Descendant (div p).
- Colors: Named (red), Hex (#FF0000), RGB (rgb(255,0,0)), RGBA (with alpha/transparency), HSL.
- Box Model: Content → Padding → Border → Margin. box-sizing: border-box is the modern default.
- Typography: font-family, font-size, font-weight, text-align, line-height, text-decoration.
- Pseudo-classes: :hover (mouse over), :focus (element focused), :active (being clicked), :visited, :nth-child.
- Order of link pseudo-classes: :link → :visited → :hover → :active (remember: LoVe HAte).
- Position: static (default), relative (offset from normal), absolute (relative to positioned parent), fixed (viewport), sticky (relative + fixed hybrid).
- z-index: Controls overlap order — higher = on top. Only works on non-static elements.
- display: none removes from layout. visibility: hidden hides but preserves space.
- Flexbox: display: flex for modern responsive layouts. justify-content, align-items, flex-wrap, gap.

End of Chapter 3 — CSS | Web Technology | Gandaki University

Prepared by: Mr. Dammar Khadayat | Pokhara, Nepal